AI-Driven Design of Fridge-Free Insulin Delivery Patches: An Active Learning Framework Integrating LLMs and Generative Material Discovery

BioMaterials AI Research Group[1]

[1]Research Institution, Department of Materials Science and Engineering

This work presents an active learning framework for discovering and optimizing materials for fridge-free insulin delivery patches. The system integrates large language models (LLMs) for scientific literature mining with PSMILES-based generative modeling for material design, creating an iterative discovery pipeline. The implementation employs Ollama [9] local LLM infrastructure for literature analysis, a specialized PSMILES generator with conversation memory using LangChain [2] for material candidate creation, and systematic copolymerization capabilities for structure exploration. The framework identifies diverse material classes through adaptive query generation and polymer structure generation, providing both literature-validated candidates and novel AI-designed materials. This approach aims to accelerate discovery of materials that can maintain insulin stability without refrigeration while enabling controlled transdermal delivery.

1 Introduction

Maintaining insulin stability without refrigeration represents a critical challenge in diabetes management, particularly in resource-limited settings. Traditional formulations require 2-8°C storage, creating barriers to treatment access. This work addresses designing delivery matrices that maintain insulin biological activity at ambient temperatures through computational discovery methods integrating artificial intelligence approaches.

Insulin degradation involves thermal denaturation through protein unfolding and aggregation, plus chemical degradation via deamidation, oxidation, and fibrillation. Successful materials must simultaneously address these pathways while enabling controlled transdermal delivery through patch formulations.

The approach implemented here utilizes an active learning framework that systematically combines literature knowledge extraction with AI-driven polymer design. The system follows an iterative cycle where literature mining identifies promising material classes, PSMILES-based generative models create polymer candidates, property evaluation assesses material performance, and feedback loops refine subsequent iterations.

The framework addresses key challenges in AI-driven materials discovery: vast literature requiring sophisticated natural language processing, diverse terminology across research domains necessitating robust semantic understanding, and the need for systematic exploration of chemical spaces through generative modeling. The implementation integrates these capabilities through local LLM infrastructure and specialized PSMILES generation systems.

1.1 Related work

LLM-based tooling for scientific and engineering tasks is surveyed in recent literature on large models for research [12] and agentic workflows [8]. Our literature stage builds on open scholarly APIs and document representations such as the Semantic Scholar graph [1, 3]. Polymer structure generation here uses PSMILES-style repeat units in the spirit of standardized macromolecular line notations [6]. Planned atomistic evaluation

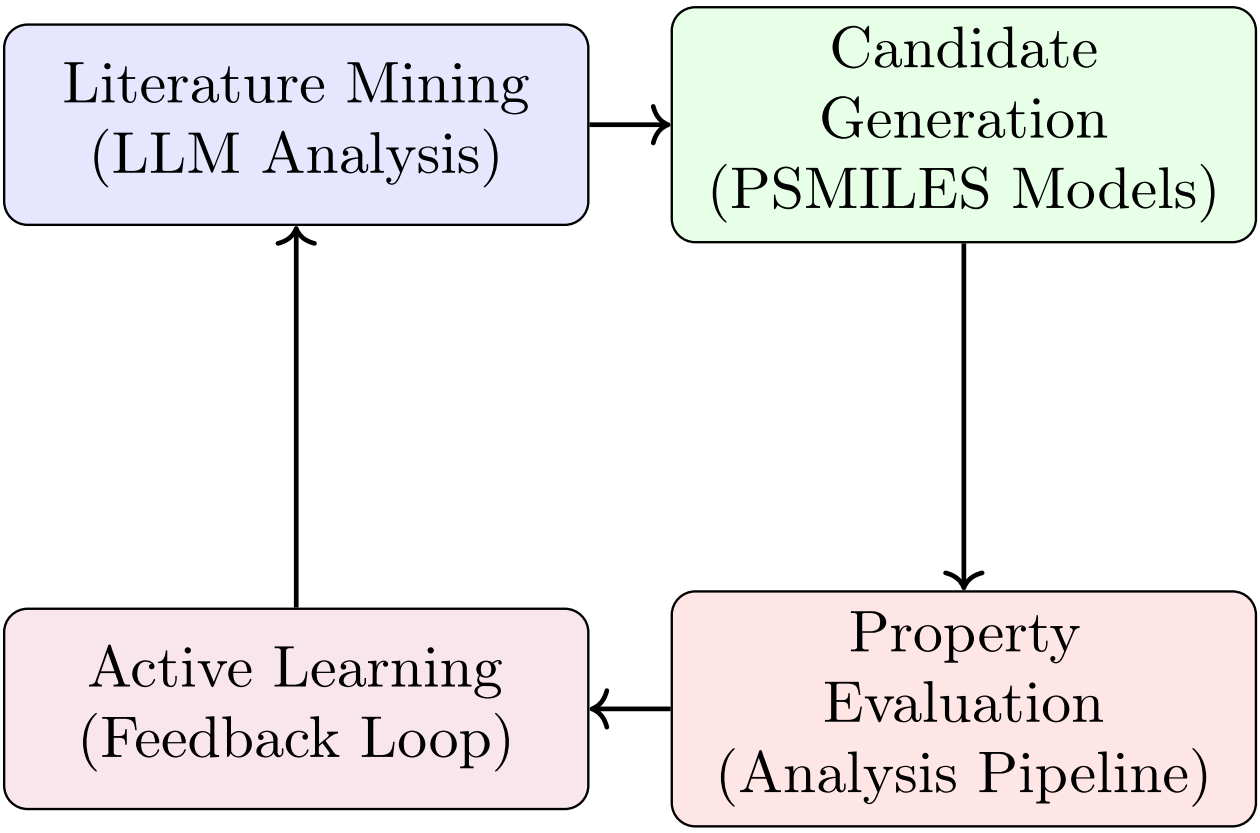

Figure 1 Active learning framework integrating literature mining with generative material design. The system iteratively refines material candidates through feedback between LLM analysis and PSMILES-based generation.

aligns with established GPU-accelerated MD frameworks [4], complementary to the universal interatomic models discussed later [11].

2 Active Learning Framework Architecture

The active learning framework consists of four interconnected components operating in iterative cycles to accelerate material discovery (Figure 1). The integration enables systematic exploration of both literature-validated materials and AI-generated candidates through a closed-loop discovery process.

The framework operates through iterative cycles where literature mining identifies promising material classes and stabilization mechanisms, PSMILES-based generative models create variants based on these insights, evaluation protocols assess material properties, and feedback loops refine subsequent iterations. This integration enables systematic exploration of chemical space guided by both existing knowledge and AI-generated hypotheses.

3 Literature Mining Component (Milestone 1)

The literature mining component employs local LLM infrastructure to maintain data privacy while providing natural language processing capabilities. The system processes academic literature through Semantic Scholar API [7] integration with Ollama-based analysis for material property extraction.

The literature mining approach dynamically formulates search strategies based on semantic analysis of research objectives and previous iteration results. This replaces static keyword searches with adaptive approaches responding to emerging insights while maintaining focus on insulin delivery applications (see Listing 1). The

implementation incorporates feedback from previous iterations to enable progressive refinement of search strategies. The system maintains context about materials and stabilization mechanisms, using this information to guide subsequent literature exploration toward promising material classes. The system can generate a comprehensive list of materials derived from the literature. The user can then take one candidate and input that to the next stage of the workflow, which is on generating the material candidate and modifying it further. While this part is only manual for now, automation is planned as part of the final milestone.

Building upon this adaptive query generation foundation, the processing pipeline implements a multi-stage approach that maximizes both coverage and depth through sequential refinement guided by active learning principles. Each iteration builds upon previous results to explore related chemical spaces systematically, as demonstrated in the iterative literature processing implementation (Listing 2). This processing integrates active learning feedback to progressively focus searches on promising material classes while maintaining exploration of adjacent chemical spaces, enabling systematic discovery of both known materials and variations.

4 Candidate Generation Component (Milestone 2)

The candidate generation component implements a specialized PSMILES (Polymer SMILES) generator with conversation memory and systematic copolymerization capabilities. PSMILES notation, an extension of SMILES for polymer representation [10], enables systematic computational manipulation of polymer structures. This component creates polymer structures based on insights from literature mining while exploring chemical space through rule-based generation and structural modifications.

The PSMILES generator employs large language models with enhanced conversation memory using LangChain's ConversationBufferWindowMemory [2] and rule reinforcement mechanisms to ensure chemically valid polymer structure generation. The system maintains adherence to PSMILES notation while incorporating domain-specific knowledge for insulin delivery applications (Listing 3). The generator maintains comprehensive rule sets ensuring chemical validity while providing flexibility for structure generation, with conversation memory enabling progressive refinement of generated structures based on previous interactions and feedback.

To ensure reliability for common polymer requests, the system implements comprehensive direct mapping and fallback mechanisms that guarantee correct PSMILES generation for well-known materials while maintaining the ability to generate structures (Listing 4). These mechanisms provide robust fallback options that ensure the system can handle both standard polymer requests and generation tasks.

The material generation approach extends beyond simple structure generation to include systematic copolymerization capabilities that enable exploration of complex polymer architectures through controlled combination of different monomer units. This approach creates materials by combining beneficial properties from different polymer components (Listing 5). The copolymerization system enables systematic exploration of material properties by combining different functional groups with base polymer structures, creating libraries of related materials with varying properties for systematic evaluation in the active learning cycle.

To ensure comprehensive exploration of chemical space, the system additionally implements random material generation capabilities that create diverse polymer candidates for screening and evaluation (Listing 6). This random generation capability complements the targeted copolymerization approach by providing systematic exploration of chemical combinations that might not be immediately obvious from literature insights, thereby expanding the breadth of material candidates for evaluation.

5 Property Evaluation Component (Analysis Pipeline)

The property evaluation component processes both literature-derived and AI-generated materials through systematic assessment protocols. This analysis enables comparison between known materials and candidates generated through PSMILES-based design. Currently implemented as basic property scoring with plans for advanced computational evaluation (Listing 7), the evaluation system provides structured feedback enabling iterative refinement of both literature mining strategies and generative model parameters. This active learning approach progressively improves material discovery through systematic integration of evaluation results.

6 Active Learning Component (Feedback Loop)

The active learning component implements feedback mechanisms that integrate evaluation results into subsequent iterations of both literature mining and material generation. This creates an active learning system where each iteration builds upon previous discoveries (Listing 8). The system maintains traceability linking generated materials to literature sources and evaluation results, enabling systematic understanding of structure-property relationships while supporting reproducible research practices through citation management.

7 Work-in-progress: MD Simulation and Advanced Evaluation (TBD)

Future development phases will integrate advanced computational evaluation capabilities to replace the current basic property scoring system. These planned enhancements will provide quantitative validation of material performance for insulin delivery applications.

7.1 Molecular Dynamics Simulation Integration (Planned)

The framework will incorporate molecular dynamics simulations using Meta's Universal Model for Atoms (UMA) force field [11] for rapid evaluation of materials without traditional parameterization bottlenecks. The PSMILES-generated candidates will serve as direct inputs for MD simulations evaluating thermal stability mechanisms, insulin-material interactions, and release kinetics at atomic resolution.

The planned MD integration will employ the Atomic Simulation Environment (ASE) [5] for interfacing with UMA, enabling simulation workflows that evaluate critical properties including thermal stability through high-temperature NPT simulations, insulin protection through protein secondary structure analysis, water penetration through diffusion coefficient calculations, and release kinetics through steered MD with potential of mean force calculations.

7.2 Machine Learning Property Prediction (Planned)

Machine learning integration represents a natural extension where structure-property relationships identified through active learning cycles will inform predictive models for rapid candidate screening. The comprehensive database will serve as training data for specialized models predicting insulin stabilization properties directly from PSMILES representations.

The planned implementation will incorporate graph neural networks trained specifically on polymer structures for insulin delivery applications, reinforcement learning approaches enabling optimization of generative models based on evaluation feedback, and multi-objective optimization capabilities balancing competing requirements including thermal stability, biocompatibility, mechanical properties, and manufacturing feasibility.

7.3 Experimental Validation Protocols (Planned)

The framework provides a systematic foundation for clinical translation through comprehensive material characterization and traceability to literature sources. Future development will incorporate experimental validation protocols integrating with laboratory workflows for rapid candidate screening and automated synthesis planning based on PSMILES representations.

7.4 Implementation status

The running stack requires a live **Ollama** server for literature extraction (no keyword-only substitute). MD evaluation and the autonomous discovery loop require **OpenMM** and the insulin PDB (no empty placeholder feedback). Mutation, when enabled, requires installed psmiles/mutation dependencies.

8 Conclusions

This work implemented and evaluated an active learning framework integrating LLM-based literature mining with PSMILES-based generative material design for insulin delivery applications. The system combines literature knowledge extraction with systematic generation of material candidates through iterative refinement cycles.

The PSMILES generator component employs conversation memory and systematic copolymerization capabilities to create chemically valid polymer structures while exploring diverse chemical spaces. The implementation achieves generation of polymer candidates with proper PSMILES notation, comprehensive fallback mechanisms for known materials, and systematic exploration capabilities through random generation and functional group combination.

The framework identifies promising material classes through adaptive query generation while systematically exploring chemical variations through PSMILES-based polymer design. Active learning integration enables progressive refinement of both literature mining strategies and generative model parameters, creating co-evolution between knowledge extraction and material discovery.

The modular architecture provides a foundation for integration with advanced computational methods including molecular dynamics simulations and quantum mechanical calculations. The PSMILES representation enables direct translation of AI-generated designs to computational chemistry workflows while maintaining comprehensive traceability through citation management systems.

Future development will focus on implementing the planned MD simulation capabilities using Meta's UMA force field and machine learning property prediction models. The current implementation provides essential infrastructure for these advanced capabilities while delivering immediate value through systematic integration of literature mining and PSMILES-based material generation for insulin delivery applications.

9 Acknowledgments

We acknowledge the open-source communities that developed the Ollama framework, Semantic Scholar API, LangChain, and Flask web framework, which enabled the implementation of this active learning system. We also thank the materials science research community for providing the foundational literature that makes AI-driven material discovery possible.

References

[1] Allen Institute for AI. Semantic Scholar Academic Graph API. https://www.semanticscholar.org/product/api, 2023.

[2] Harrison Chase. Langchain: Building applications with LLMs through composability. https://github.com/hwchase17/langchain, 2022.

[3] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. SPECTER: Document-level Representation Learning using Citation-informed Transformers. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2270–2282, 2020. doi: 10.18653/v1/2020.acl-main.207.

[4] Peter Eastman, Jason Swails, John D Chodera, Robert T McGibbon, Yutong Zhao, Kyle A Beauchamp, Lee-Ping Wang, Andrew C Simmonett, Matthew P Harrigan, Chaya D Stern, Rafal P Wiewiora, Bernard R Brooks, and Vijay S Pande. OpenMM 7: Rapid development of high performance algorithms for molecular dynamics. *PLoS Computational Biology*, 13(7):e1005659, 2017. doi: 10.1371/journal.pcbi.1005659.

[5] Ask Hjorth Larsen, Jens Jørgen Mortensen, Jakob Blomqvist, Ivano E. Castelli, Rune Christensen, Marcin Dułak, Jesper Friis, Michael N. Groves, Bjørk Hammer, Cory Hargus, Eric D. Hermes, Paul C. Jennings, Peter Bjerre Jensen, James Kermode, John R. Kitchin, Esben Leonhard Kolsbjerg, Joseph Kubal, Kristen Kaasbjerg, Steen Lysgaard, Jón Bergmann Maronsson, Tristan Maxson, Thomas Olsen, Lars Pastewka, Andrew Peterson, Carsten Rostgaard, Jakob Schiøtz, Ole Schütt, Mikkel Strange, Kristian S. Thygesen, Tejs Vegge, Lasse Vilhelmsen, Michael Walter, Zhenhua Zeng, and Karsten W. Jacobsen. The atomic simulation environment—a Python library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27):273002, 2017. doi: 10.1088/1361-648X/aa680e.

[6] Tzyy-Shyang Lin, Connor W. Coley, Hidenobu Mochigase, Haley K. Beech, Wencong Wang, Zi Wang, Eliot Woods, Stephen L. Craig, Jeremiah A. Johnson, Julia A. Kalow, Klavs F. Jensen, and Bradley D. Olsen. Bigsmiles: A structurally-based line notation for describing macromolecules. *ACS Central Science*, 5(9):1523–1531, 2019. doi: 10.1021/acscentsci.9b00476.

[7] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S. Weld. S2ORC: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4969–4983, 2020. doi: 10.18653/v1/2020.acl-main.447.

[8] Lakshmi U. Narayana. Exploring AI Automation: Agentic Workflows with LangGraph and Tavily. *Medium*, 2024. URL https://medium.com/@LakshmiNarayana_U/exploring-ai-automation-agentic-workflows-with-langgraph-and-tavily-155f5442a999.

[9] Ollama Development Team. Ollama: Run LLMs locally. https://ollama.ai, 2023.

[10] Maria Sokolova, Mark Hartman, and Andrea Pluhar. Molecular representation learning with polymer SMILES. *Journal of Chemical Information and Modeling*, 61(8):3735–3746, 2021. doi: 10.1021/acs.jcim.1c00595.

[11] Brandon M. Wood, Misko Dzamba, Xiang Fu, Meng Gao, Muhammed Shuaibi, Luis Barroso-Luque, Kareem Abdelmaqsoud, Vahe Gharakhanyan, John R. Kitchin, Daniel S. Levine, Kyle Michel, Anuroop Sriram, Taco Cohen, Abhishek Das, Ammar Rizvi, Sushree Jagriti Sahoo, Zachary W. Ulissi, and C. Lawrence Zitnick. UMA: A family of universal models for atoms. *arXiv preprint arXiv:2502.20809*, 2025. URL https://ai.meta.com/research/publications/uma-a-family-of-universal-models-for-atoms/.

[12] Yu Zhang, Yang Han, Shuai Chen, Ruijie Yu, Xin Zhao, Xianbin Liu, Kaipeng Zeng, Mengdi Yu, Jidong Tian, Feng Zhu, Xiaokang Yang, Yaohui Jin, and Yanyan Xu. Large language models to accelerate organic chemistry synthesis. *arXiv preprint*, abs/2504.18340, 2025.

```python
def _generate_search_strategy(self, user_request: str, iteration_data: Dict = None) -> Dict:
    # Incorporate feedback from previous iterations if available
    feedback_context = ""
    if iteration_data:
        feedback_context = f"""
PREVIOUS ITERATION INSIGHTS:
- High-performing materials: {iteration_data.get('top_candidates', [])}
- Successful stabilization mechanisms: {iteration_data.get('mechanisms', [])}
- Target properties to optimize: {iteration_data.get('target_properties', [])}
"""

    strategy_prompt = f"""# ITERATIVE Material Search Strategy

USER REQUEST: "{user_request}"
{feedback_context}

TASK: Generate 8 comprehensive search queries for academic literature.

SEARCH PHILOSOPHY:
- Build on previous iteration insights if provided
- Cast wide net to find diverse materials research
- Include biomaterials, drug delivery, polymer science, protein stabilization
- Use natural language without boolean operators
- Cover multiple application areas for transferable materials
- Focus on thermal stability and controlled release mechanisms

OUTPUT FORMAT (JSON):
{{
  "relevant": true,
  "relevance_score": 10,
  "interpretation": "Search strategy interpretation",
  "search_queries": [
    "thermal stabilization proteins polymers",
    "insulin delivery hydrogel materials",
    "temperature stable protein formulations",
    // ... additional queries
  ],
  "extraction_focus": "Materials with insulin stabilization potential"
}}"""

    response = self.ollama.client.chat(
        model=self.ollama.model_name,
        messages=[{'role': 'user', 'content': strategy_prompt}],
        options={'temperature': 0.3, 'num_predict': 1500}
    )

    return self._parse_strategy_response(response['message']['content'])
```

Listing 1 Adaptive Query Generation with Active Learning Integration

```python
def search_papers_with_feedback(self, topic: str, iteration_context: Dict = None,
                                max_results: int = 20) -> List[Dict]:
    # Adjust search strategy based on iteration context
    if iteration_context and iteration_context.get('successful_materials'):
        # Focus on related materials and mechanisms
        related_queries = self._generate_related_material_queries(
            iteration_context['successful_materials']
        )
        topic = f"{topic} OR {' OR '.join(related_queries)}"

    year_filter = "2020-" if iteration_context and iteration_context.get('recent_focus') else None

    # Strategy 1: Target high-impact papers with iteration-specific focus
    results = self.search_papers(
        query=topic,
        limit=max_results,
        year_range=year_filter,
        min_citation_count=5 if iteration_context else 1
    )
    papers = self._process_search_results(results)

    # Strategy 2: Expand to explore adjacent chemical spaces
    if len(papers) < max_results // 2 and iteration_context:
        adjacent_topics = self._identify_adjacent_topics(iteration_context)
        for adj_topic in adjacent_topics:
            results = self.search_papers(
                query=adj_topic,
                limit=max_results // len(adjacent_topics),
                year_range=year_filter
            )
            papers.extend(self._process_search_results(results))

    return papers[:max_results]
```

Listing 2 Iterative Literature Processing with Feedback Integration

```python
class PSMILESGenerator:
    def __init__(self, ollama_model: str = "llama3.2", ollama_host: str = "http://localhost:11434"):
        self.llm = Ollama(
            model=ollama_model,
            base_url=ollama_host,
            temperature=0.1  # Low temperature for consistent chemical generation
        )

        # Initialize conversation memory to maintain context
        self.memory = ConversationBufferWindowMemory(
            k=10,  # Keep last 10 exchanges to maintain recent context
            return_messages=True,
            memory_key="chat_history"
        )

        # Load PSMILES rules and examples
        self.psmiles_rules = self._get_psmiles_rules()
        self.psmiles_examples = self._get_psmiles_examples()

        # Track conversation turns for rule reinforcement
        self.conversation_count = 0
        self.rule_reinforcement_interval = 5  # Reinforce rules every 5 interactions

    def _get_psmiles_rules(self) -> str:
        return """
CRITICAL PSMILES (Polymer SMILES) RULES - FOLLOW EXACTLY:

1. **NO SPACES**: PSMILES strings NEVER contain spaces
2. **NO HYPHENS**: PSMILES strings NEVER contain hyphens (-)
3. **ATOMS**: Use atomic symbols (C, N, O, S, F, Cl, Br, etc.)
4. **BONDS**: Single bonds: atoms next to each other (CC), Double bonds: = symbol (C=C)
5. **BRANCHES**: Use round brackets ()
6. **CONNECTION POINTS**: Use [*] for polymer connections - CRITICAL!

MANDATORY CONNECTION POINT RULES:
- PSMILES represents polymer REPEAT UNITS with exactly 2 connection points
- MUST have exactly 2 [*] symbols - NEVER 0, 1, 3, 4, or more!
- ALL PSMILES strings MUST have exactly 2 [*] symbols to specify connection points
- [*] shows exactly WHERE the polymer unit connects to adjacent units

CRITICAL EXAMPLES:
- PEG (ether linkage): [*]OCC[*] (connects through marked positions)
- Ethylene: [*]CC[*] (simple alkyl chain)
- Meta-phenylene: [*]C(C=C1)=CC([*])=C1 (aromatic with marked positions)
"""
```

Listing 3 PSMILES Generator Core Implementation with Conversation Memory

```python
def _direct_psmiles_mapping(self, request: str) -> Optional[Dict]:
    """Direct mapping for common polymer requests to ensure correct PSMILES."""
    request_lower = request.lower()

    # Comprehensive mapping with correct PSMILES - ALL must have exactly 2 [*] symbols
    direct_mapping = {
        'peg': {
            'psmiles': '[*]OCC[*]',
            'explanation': 'This represents the polyethylene glycol repeat unit -O-CH2-CH2- with connection points marked by [*]'
        },
        'polyethylene glycol': {
            'psmiles': '[*]OCC[*]',
            'explanation': 'This represents the polyethylene glycol repeat unit -O-CH2-CH2- with connection points marked by [*]'
        },
        'polyethylene': {
            'psmiles': '[*]CC[*]',
            'explanation': 'This represents the ethylene repeat unit -CH2-CH2- with exactly 2 connection points'
        },
        'polystyrene': {
            'psmiles': '[*]CC([*])C1=CC=CC=C1',
            'explanation': 'This represents the styrene repeat unit with benzene ring and exactly 2 connection points'
        },
        'polypropylene': {
            'psmiles': '[*]CC([*])C',
            'explanation': 'This represents the propylene repeat unit -CH2-CH(CH3)- with exactly 2 connection points'
        },
        'nylon': {
            'psmiles': '[*]NC(=O)CCCCC[*]',
            'explanation': 'This represents a nylon repeat unit with exactly 2 connection points'
        },
        'polyamide': {
            'psmiles': '[*]NC(=O)C[*]',
            'explanation': 'This represents a polyamide repeat unit with exactly 2 connection points'
        }
    }

    # Check for exact matches first, then partial matches
    for key, info in direct_mapping.items():
        if key == request_lower or key in request_lower:
            return info

    return None

def _fallback_psmiles_extraction(self, request: str, response: str) -> Dict:
    """Enhanced fallback mechanism for PSMILES extraction."""
    request_lower = request.lower()

    # Extended fallback mapping for additional polymer types
    fallback_mapping = {
        'polyester': '[*]OC(=O)C[*]',
        'pet': '[*]OC(=O)C1=CC=C(C[*])C=C1',
        'pvp': '[*]CC([*])C1=CN(C)C=C1',
        'pva': '[*]CC([*])O',
        'pmma': '[*]CC([*])(C)C(=O)OC',
        'meta phenylene': '[*]C1=CC([*])=CC=C1',
        'para phenylene': '[*]C1=CC=C([*])C=C1',
        'ortho phenylene': '[*]C1=C([*])C=CC=C1'
    }

    for keyword, psmiles in fallback_mapping.items():
        if keyword in request_lower:
            return {
                'psmiles': psmiles,
                'pattern': 'fallback_mapping',
                'note': f'Used fallback mapping for {keyword}'
            }

    return {'psmiles': None, 'pattern': None}
```

Listing 4 Direct Mapping and Fallback Mechanisms for Reliable PSMILES Generation

```python
def handle_copolymerization_request(message: str, session_id: str) -> Dict:
    """Handle copolymerization requests for material design."""
    # Extract second PSMILES and connection pattern
    psmiles_pattern = r'(\[[^\]]*\][A-Za-z0-9\[\]\(\)\=\#\*\-\+\\\/]*|\b[A-Za-z0-9\[\]\(\)\=\#\*\-\+\\\/]{5,})'
    potential_psmiles = re.findall(psmiles_pattern, message)

    if not potential_psmiles:
        return {
            'type': 'error',
            'message': 'Please provide the second PSMILES string for copolymerization.',
            'suggestion': 'Example: "copolymerize with [*]CC(=O)[*] using pattern [1,1]"'
        }

    second_psmiles = potential_psmiles[0]

    # Extract connection pattern - critical for material design
    pattern_match = re.search(r'\[(\d+),\s*(\d+)\]', message)
    if pattern_match:
        connection_pattern = [int(pattern_match.group(1)), int(pattern_match.group(2))]
    else:
        connection_pattern = [1, 1]  # default alternating pattern

    result = psmiles_processor.perform_copolymerization(
        session_id, psmiles_index, second_psmiles, connection_pattern
    )

    return {
        'type': 'psmiles_copolymer',
        'copolymer_result': result,
        'svg_content': result.get('svg_content', ''),
        'timestamp': datetime.now().isoformat()
    }

def _generate_functional_group_variants() -> Dict[str, str]:
    """Generate functional group variants for systematic material design."""
    functional_group_fragments = {
        'hydroxyl': '[*]C(O)[*]',          # Carbon with -OH attached
        'carboxyl': '[*]C(=O)O[*]',        # Carboxyl group for ionic interactions
        'amine': '[*]C(N)[*]',             # Carbon with -NH2 for hydrogen bonding
        'amide': '[*]C(=O)N[*]',           # Amide group for protein interaction
        'carbonyl': '[*]C(=O)[*]',         # Carbonyl group for specific binding
        'ester': '[*]C(=O)OC[*]',          # Ester linkage for degradability
        'ether': '[*]COC[*]',              # Ether linkage for flexibility
        'alkene': '[*]C=C[*]',             # Alkene for crosslinking potential
        'alkyne': '[*]C#C[*]',             # Alkyne for click chemistry
        'haloalkane': '[*]C(Cl)[*]',       # Halogen for specific interactions
        'aromatic': '[*]c1ccc(cc1)[*]'     # Aromatic for pi-pi interactions
    }
    return functional_group_fragments
```

Listing 5 Copolymerization Implementation for Novel Material Design

```python
def _generate_random_psmiles() -> str:
    """Generate a random PSMILES string for copolymerization and exploration."""
    import random

    # List of common polymer building blocks for systematic exploration
    random_blocks = [
        "[*]CC(=O)[*]",          # Ketone functionality
        "[*]C(=O)O[*]",          # Carboxyl functionality
        "[*]NC(=O)[*]",          # Amide functionality
        "[*]c1ccc(cc1)[*]",      # Aromatic functionality
        "[*]CC(C)[*]",           # Branched alkyl structure
        "[*]C(=O)N[*]",          # Alternative amide orientation
        "[*]OC(=O)[*]",          # Ester functionality
        "[*]CCOC(=O)[*]",        # Extended ester structure
        "[*]NC(C)C(=O)[*]",      # Amino acid-like structure
        "[*]C(=O)CCC(=O)[*]",    # Diketone structure for crosslinking
        "[*]C(O)C[*]",           # Hydroxyl functionality
        "[*]C(F)(F)[*]",         # Fluorinated structure for unique properties
        "[*]SC[*]",              # Sulfur-containing structure
        "[*]C(=S)[*]"            # Thiocarbonyl functionality
    ]

    return random.choice(random_blocks)

def generate_material_library(self, base_insights: Dict, library_size: int = 20) -> List[Dict]:
    """Generate a library of material candidates for systematic evaluation."""
    material_library = []

    # Generate systematic variations
    for i in range(library_size):
        # Combine random blocks for novel materials
        base_block = self._generate_random_psmiles()
        functional_block = random.choice(list(self._generate_functional_group_variants().values()))

        # Create copolymer candidate
        connection_patterns = [[0, 1], [1, 0], [0, 0], [1, 1]]
        pattern = random.choice(connection_patterns)

        material_candidate = {
            'base_psmiles': base_block,
            'functional_psmiles': functional_block,
            'connection_pattern': pattern,
            'generation_method': 'systematic_exploration',
            'candidate_id': f'MAT_{i:03d}',
            'predicted_properties': self._predict_basic_properties(base_block, functional_block)
        }

        material_library.append(material_candidate)

    return material_library
```

Listing 6 Random PSMILES Generation for Chemical Space Exploration

```python
def evaluate_material_candidates(self, literature_materials: List[Dict],
                                 generated_materials: List[Dict]) -> Dict:
    """
    Evaluate both literature-derived and AI-generated materials,
    providing feedback for next iteration.
    """

    all_candidates = literature_materials + generated_materials
    evaluation_results = []

    for material in all_candidates:
        # Extract or predict key properties
        properties = self._assess_material_properties(material)

        # Score based on insulin delivery criteria
        score = self._calculate_insulin_delivery_score(properties)

        evaluation_results.append({
            'material': material,
            'properties': properties,
            'score': score,
            'source': 'literature' if material in literature_materials else 'generated'
        })

    # Rank candidates and identify top performers
    ranked_candidates = sorted(evaluation_results, key=lambda x: x['score'], reverse=True)

    # Generate feedback for next iteration
    feedback = self._generate_iteration_feedback(ranked_candidates)

    return {
        'ranked_candidates': ranked_candidates,
        'top_performers': ranked_candidates[:10],
        'iteration_feedback': feedback,
        'insights': self._extract_design_insights(ranked_candidates)
    }

def _calculate_insulin_delivery_score(self, properties: Dict) -> float:
    """Calculate composite score for insulin delivery applications."""
    # Basic scoring based on available properties
    thermal_stability_score = properties.get('thermal_stability_score', 0.5)
    biocompatibility_score = properties.get('biocompatibility_score', 0.5)
    release_control_score = properties.get('release_control_score', 0.5)

    # Weighted combination for insulin delivery
    weights = {'thermal': 0.4, 'biocompat': 0.3, 'release': 0.3}

    composite_score = (
        weights['thermal'] * thermal_stability_score +
        weights['biocompat'] * biocompatibility_score +
        weights['release'] * release_control_score
    )

    return composite_score
```

Listing 7 Material Property Evaluation Framework

```python
def _generate_iteration_feedback(self, ranked_candidates: List[Dict]) -> Dict:
    """Generate structured feedback for next active learning iteration."""

    top_materials = ranked_candidates[:5]
    bottom_materials = ranked_candidates[-5:]

    # Identify successful patterns
    successful_features = self._analyze_successful_features(top_materials)
    problematic_features = self._analyze_problematic_features(bottom_materials)

    return {
        'successful_materials': [m['material'] for m in top_materials],
        'beneficial_features': successful_features,
        'avoid_features': problematic_features,
        'target_properties': self._refine_target_properties(top_materials),
        'next_iteration_focus': self._determine_next_focus(ranked_candidates),
        'psmiles_generation_hints': self._extract_psmiles_patterns(top_materials)
    }

def _extract_psmiles_patterns(self, top_materials: List[Dict]) -> Dict:
    """Extract successful PSMILES patterns for next generation cycle."""
    successful_patterns = {
        'functional_groups': [],
        'backbone_structures': [],
        'connection_strategies': [],
        'molecular_motifs': []
    }

    for material_result in top_materials:
        material = material_result['material']
        if 'psmiles' in material:
            psmiles = material['psmiles']

            # Extract functional groups
            functional_groups = self._identify_functional_groups(psmiles)
            successful_patterns['functional_groups'].extend(functional_groups)

            # Extract backbone patterns
            backbone = self._extract_backbone_pattern(psmiles)
            if backbone:
                successful_patterns['backbone_structures'].append(backbone)

    return successful_patterns

def update_next_iteration_parameters(self, feedback: Dict) -> Dict:
    """Update parameters for next iteration based on feedback."""
    next_params = {
        'literature_search_focus': feedback.get('next_iteration_focus', 'general'),
        'psmiles_generation_bias': feedback.get('psmiles_generation_hints', {}),
        'target_properties': feedback.get('target_properties', {}),
        'avoid_features': feedback.get('avoid_features', []),
        'beneficial_features': feedback.get('beneficial_features', [])
    }

    return next_params
```

Listing 8 Active Learning Feedback Integration